



Informe Post Mortem Aplicación EntregaYa

Autores

Andrés Castrillo, Esteban Chaustre, Alejandro Martinez, Daniel Barreto, Juan Arbelaez

Ingeniería de Sistemas

Presentado a: Luis Moreno Sandoval

Fecha de entrega: 19 de mayo 2026

1. Introducción

Este informe post mortem documenta el desarrollo completo de la aplicación EntregaYa, un proyecto académico de gestión de trabajos colaborativos desarrollado a lo largo de 14 sprints entre febrero y mayo de 2026. El equipo conformado por cinco estudiantes aplicó metodologías ágiles de manera rigurosa, logrando completar 54 historias de usuario distribuidas en 7 hitos principales.

El presente documento tiene como objetivo analizar cuantitativamente el proceso de desarrollo, evidenciar la participación equitativa del equipo, identificar patrones de trabajo, y extraer lecciones aprendidas que fortalezcan nuestras competencias. A través de métricas objetivas de commits, distribución de tareas, y retrospectivas estructuradas, buscamos demostrar el cumplimiento de objetivos y también reflexionar sobre nuestro desempeño como equipo de desarrollo.

2. Hitos finalizados según lo planificado

La planificación del proyecto se estructuró en torno a 7 hitos temáticos que representaban entregables funcionales completos. Cada hito agrupaba historias de usuario relacionadas y se distribuía a lo largo de distintos sprints según la complejidad técnica y las dependencias entre funcionalidades.

Milestone	Objetivo	Sprints	Estado	HU asociadas
Version 1.0 - Entregar Documentación	Definir la problemática, justificación, alcance y documentación inicial del proyecto de software.	1	Finalizado	Ninguna
Módulo Inicio de Sesión, dokerizar y conectar base de datos	Desarrollar las funcionalidades iniciales de autenticación, gestión básica de trabajos y tareas, integrando PostgreSQL y una implementación temprana de Docker.	2 - 4	Finalizado	HU01-HU04
Creación de roles en Trabajos	Implementar un sistema de roles y permisos para controlar las acciones de los usuarios dentro de cada trabajo.	5 - 6	Finalizado	HU05 - HU14
Edición, comunicación y visualización de trabajos y tareas	Mejorar la interacción entre usuarios mediante edición de tareas, visualización de	7	Finalizado	HU15 - HU19

	información y sistema de notificaciones.			
Implementar Patrones GoF	Aplicar patrones de diseño GoF para mejorar la organización, mantenibilidad y escalabilidad de la arquitectura del proyecto.	8 - 9	Finalizado	HU20 - HU29
Segunda Entrega	Consolidar nuevas funcionalidades del sistema e integrar pipelines iniciales de CI/CD para automatizar validaciones y despliegues básicos.	10 - 11	Finalizado	HU30 - HU39
Calidad y Métricas	Implementar pruebas, métricas y herramientas de calidad para asegurar estabilidad, cobertura y automatización completa del proyecto.	12 - 14	Finalizado	HU40 - HU54

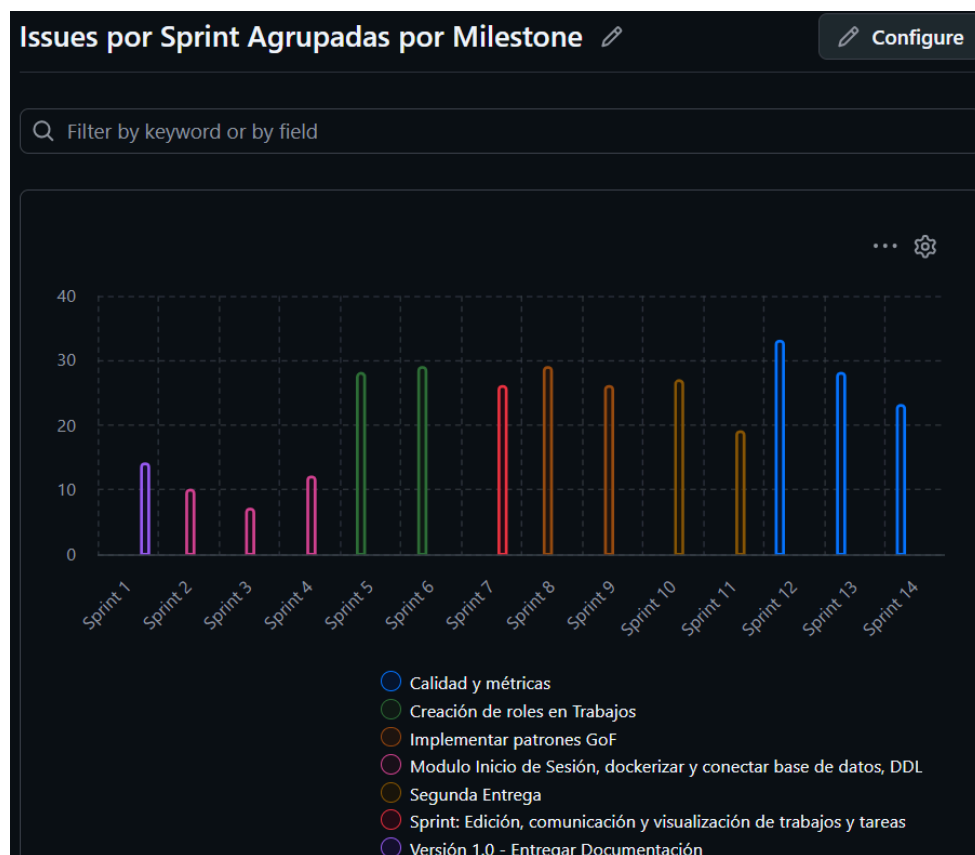


Figura 1. Número de Issues asignados por sprint agrupados por Milestone

La Figura 1 evidencia una distribución variable pero controlada del trabajo a través de los sprints. Se observan picos de actividad en los sprints 5, 6, 12 y 13, correspondientes a hitos de mayor complejidad t cnica como "Creaci n de roles en Trabajos" y "Calidad y m tricas". Esta variabilidad no refleja desorganizaci n, sino una priorizaci n: los sprints con menor cantidad de issues (1-4) se enfocan en tareas de infraestructura, configuraci n inicial y documentaci n, mientras que los sprints de mayor densidad implementaron funcionalidades del sistema.

M�trica	Resultado
Total de hitos	7
Hitos completados	7
Porcentaje de cumplimiento	100%

Algo a destacar es que no contamos con hitos incompletos o abandonados, lo que demuestra una estimaci n razonable y buena capacidad de ajustarnos durante la implementaci n y ejecuci n de estos.



Figura 2. Hitos definidos a lo largo del desarrollo del proyecto

← Back to Milestones

Versión 1.0 - Entregar Documentación

Closed Due by February 13, 2026 • Closed 4 days ago

- Aprender sobre cómo usar GitHub.
- Crear el documento soporte con evidencias.
- Entregar el documento en campus virtual.

100% complete

Open 0 Closed 14

- ☐ ☒ Definir requisitos **diseño**
 Task #3 · by andresj-castrillo was closed on Feb 13
- ☐ ☒ Definir propuesta de valor **diseño**
 Task #2 · by andresj-castrillo was closed on Feb 13
- ☐ ☒ Definición de roles **diseño**
 Task #4 · by andresj-castrillo was closed on Feb 13
- ☐ ☒ Creación de Mockups **front**
 Task #5 · by andresj-castrillo was closed on Feb 17
- ☐ ☒ Organizar repositorio **pruebas**
 Task #6 · by andresj-castrillo was closed on Feb 10
- ☐ ☒ Sprint Planning
 Task #7 · by andresj-castrillo was closed on Feb 11

Activar Windows
Ve a Configuración para activar Windows

Figura 3. Milestone Version 1.0 - Entregar Documentación

← Back to Milestones

Modulo Inicio de Sesión, dockerizar y conectar base de datos, DDL

Closed Due by March 7, 2026 • Closed 2 months ago

100% complete

Open 0 Closed 31

- ☐ ☒ Sprint Planning 4 **documentation**
 Task #95 · by andresj-castrillo was closed on Mar 2
- ☐ ☒ Sprint Planning 4 **documentation**
 Task #96 · by andresj-castrillo was closed on Mar 2 · Duplicate
- ☐ ☒ Desarrollo de la interfaz de login **front**
 Task #23 · by andresj-castrillo was closed on Feb 20
- ☐ ☒ Daily 1 Sprint 2 **documentation**
 Task #27 · by andresj-castrillo was closed on Feb 17
- ☐ ☒ Daily 2 Sprint 2 **documentation**
 Task #29 · by andresj-castrillo was closed on Feb 19
- ☐ ☒ Modulo registro e inicio de sesión **4 / 4** **despliegue y documentación** **front** **pruebas**
 Feature #21 · by andresj-castrillo was closed on Feb 23
- ☐ ☒ Implementación servicio de autenticación **despliegue y documentación**
 Task #24 · by andresj-castrillo was closed on Feb 20

Activar Windows
Ve a Configuración para activar Windows

Figura 4. Milestone Módulo Inicio de Sesión, dockerizar y conectar base de datos, DDL

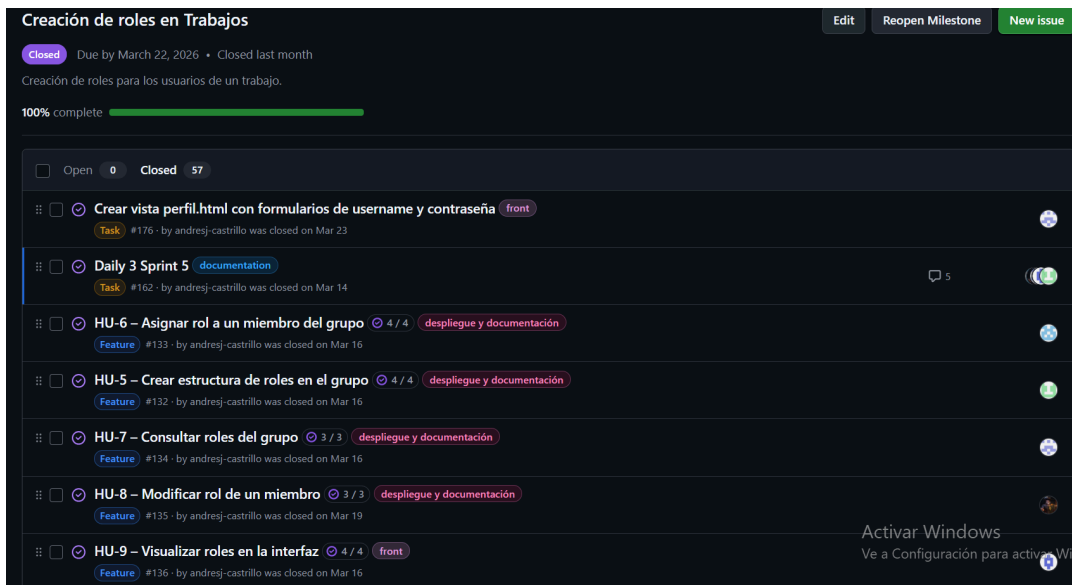


Figura 5. Milestone Creación de roles en Trabajos

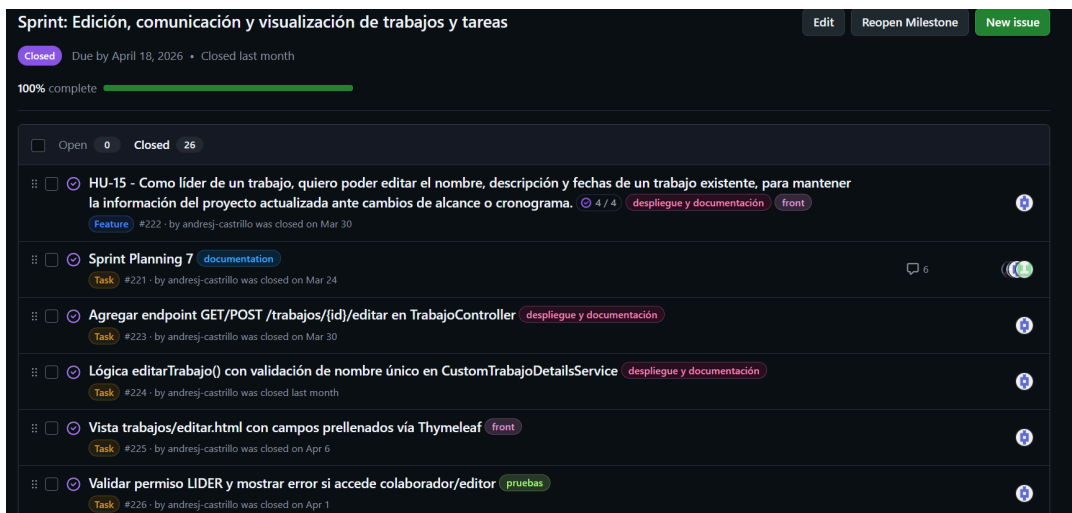


Figura 6. Milestone Edición, comunicación y visualización de trabajos y tareas

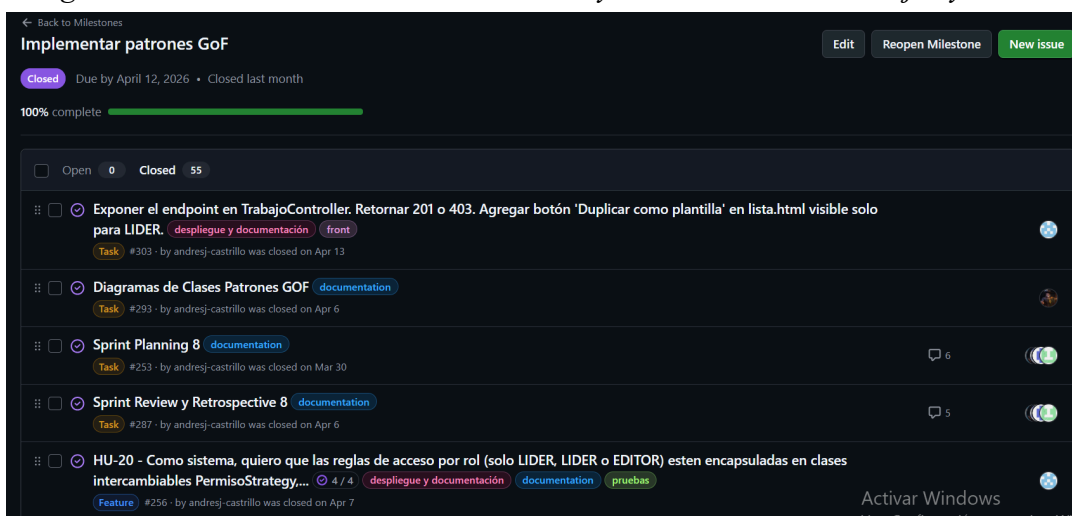


Figura 7. Milestone Implementar Patrones GoF

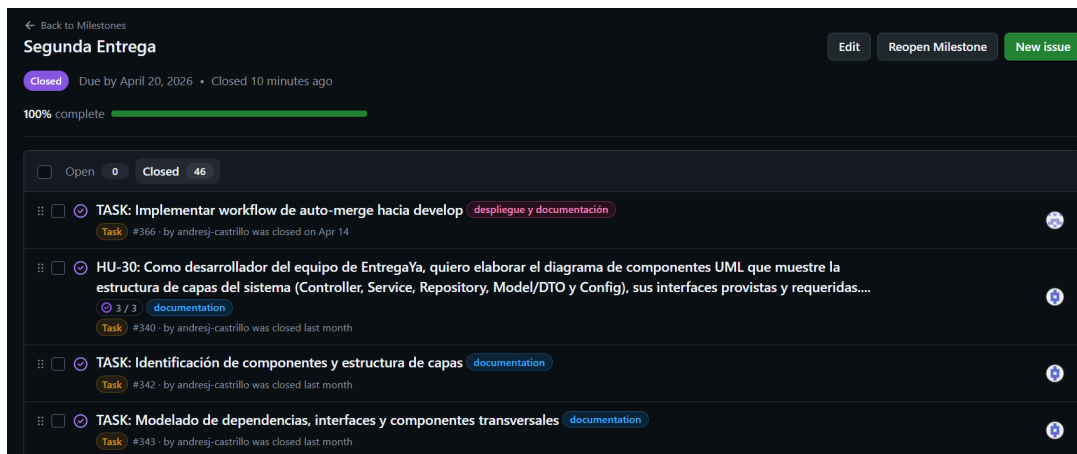


Figura 8. Milestone Segunda Entrega



Figura 9. Milestone Calidad y métricas

El milestone "Calidad y métricas" merece análisis especial por su impacto directo en el Sprint 14. Este milestone obligó al equipo a adoptar prácticas de ingeniería profesional: cobertura de pruebas >80% con JaCoCo, análisis estático con SonarQube, pipelines CI/CD funcionales, y métricas de latencia. La exigencia de estas prácticas provocó un volumen inusual de commits (126 en Sprint 14 vs. promedio de 49.7), explicado por tres factores: (1) correcciones iterativas de code smells detectados por SonarQube, (2) creación masiva de archivos de test para alcanzar cobertura objetivo, y (3) ajustes múltiples en workflows de GitHub Actions hasta lograr ejecuciones exitosas. El resultado final fue un sistema robusto y mantenible que superó los estándares mínimos.

3. Historias de Usuario terminadas y trazables en el tiempo

HU	Descripción	Sprint	Milestone
HU01	Como estudiante universitario quiero registrarme e iniciar sesión de manera sencilla para acceder a la plataforma sin necesidad de conocimientos técnicos previos.	2	Módulo Inicio de Sesión, dokerizar y conectar base de datos
HU02	Como estudiante, quiero crear un Trabajo con los campos id, titulo, descripcion, fecha de entrega y fecha de creación, para almacenar y gestionar la información de los trabajos académicos.	3	Módulo Inicio de Sesión, dokerizar y conectar base de datos
HU03	Como desarrollador, quiero implementar las capas de repositorio, servicio y controlador para la funcionalidad de creación de trabajos e invitaciones, para garantizar una arquitectura organizada.	3	Módulo Inicio de Sesión, dokerizar y conectar base de datos
HU04	Como estudiante quiero visualizar el progreso de mi trabajo para saber qué tareas están pendientes o finalizadas.	4	Módulo Inicio de Sesión, dokerizar y conectar base de datos
HU05	Como sistema quiero tener una estructura de roles dentro de los grupos para poder asignar responsabilidades a los estudiantes.	5	Creación de roles en Trabajos
HU06	Como estudiante quiero asignar un rol a un miembro del grupo para definir su responsabilidad dentro del trabajo.	5	Creación de roles en Trabajos
HU07	Como estudiante quiero consultar los roles de los miembros del grupo para entender las responsabilidades de cada integrante.	5	Creación de roles en Trabajos
HU08	Como estudiante quiero modificar el rol de un miembro del grupo para ajustar las responsabilidades cuando sea necesario.	5	Creación de roles en Trabajos
HU09	Como estudiante quiero visualizar los roles de cada integrante del grupo en la interfaz para identificar sus responsabilidades.	5	Creación de roles en Trabajos
HU10	Como usuario registrado, quiero poder actualizar mi nombre de usuario y cambiar mi contraseña desde una página de perfil, para mantener mis credenciales seguras y actualizadas sin necesidad de contactar a nadie.	6	Creación de roles en Trabajos

HU11	Como colaborador de un trabajo, quiero ver indicadores visuales cuando una tarea esté vencida o a menos de 24 horas de su fecha límite, para poder priorizar mi trabajo y no perder entregas importantes.	6	Creación de roles en Trabajos
HU12	Como editor o líder de un trabajo, quiero poder editar el nombre, descripción, dificultad, fechas y responsables de una tarea ya creada, para corregir errores o ajustar el alcance sin tener que borrar la tarea y crearla de nuevo.	6	Creación de roles en Trabajos
HU13	Como editor o líder de un trabajo, quiero poder editar el nombre, descripción, dificultad, fechas y responsables de una tarea ya creada, para corregir errores o ajustar el alcance sin tener que borrar la tarea y crearla de nuevo.	6	Creación de roles en Trabajos
HU14	Como líder de un trabajo, quiero poder expulsar a un colaborador del trabajo, para mantener el equipo actualizado cuando alguien ya no deba tener acceso al proyecto.	6	Creación de roles en Trabajos
HU15	Como líder de un trabajo, quiero poder editar el nombre, descripción y fechas de un trabajo existente, para mantener la información del proyecto actualizada ante cambios de alcance o cronograma.	7	Edición, comunicación y visualización de trabajos y tareas
HU16	Como usuario registrado, quiero tener un correo electrónico asociado a mi cuenta, para poder ser contactado por mis colaboradores y recibir información relevante de mis trabajos.	7	Edición, comunicación y visualización de trabajos y tareas
HU17	Como colaborador de un trabajo, quiero poder buscar tareas por nombre y filtrarlas por estado o dificultad, para encontrar rápidamente las tareas relevantes en proyectos con muchas tareas.	7	Edición, comunicación y visualización de trabajos y tareas
HU18	Como colaborador de un trabajo, quiero poder dejar comentarios en las tareas, para comunicar avances, dudas o aclaraciones directamente en el contexto de cada tarea sin necesidad de canales externos.	7	Edición, comunicación y visualización de trabajos y tareas
HU19	Como líder de un trabajo, quiero visualizar las tareas del trabajo en un calendario mensual, para planificar mejor los plazos de entrega e identificar de un vistazo semanas con alta carga de trabajo.	7	Edición, comunicación y visualización de trabajos y tareas
HU20	Como sistema, quiero que las reglas de acceso por rol estén encapsuladas en clases intercambiables PermisoStrategy, para eliminar los bloques de lógica duplicados que hoy existen en	8	Implementar Patrones GoF

	CustomTrabajoDetailsService, CustomInvitacionDetailsService y TareaController.		
HU21	Como desarrollador del equipo, quiero una clase TareaBuilder que permite construir objetos Tarea de forma encadenada y que valide que el nombre no este vacio y que la fecha final no sea anterior a la inicial antes de crear el objeto, para prevenir tareas con datos inconsistentes desde el origen.	8	Implementar Patrones GoF
HU22	Como líder de un trabajo, quiero que al crear o editar una tarea el sistema use TareaBuilder para construir el objeto con validaciones automáticas, para que los errores de datos se detecten antes de llegar a la base de datos y el mensaje de error llegue al formulario de forma clara.	8	Implementar Patrones GoF
HU23	Como desarrollador, quiero una clase DashboardFacade que encapsule los cálculos de trabajos activos, progresos, próximos a vencer, tareas vencidas e invitaciones pendientes, para sacar esa lógica del controlador y exponerla a través de un único punto de acceso.	8	Implementar Patrones GoF
HU24	Como desarrollador quiero que AuthController solo tenga una dependencia (DashboardFacade) para que el método dashboard() no supere 5 líneas de código.	8	Implementar Patrones GoF
HU25	Como líder de un trabajo, quiero duplicar un trabajo existente como plantilla, para crear rápidamente nuevos proyectos con la misma estructura sin comenzar desde cero.	9	Implementar Patrones GoF
HU26	Como colaborador con rol EDITOR o LÍDER dentro de un trabajo, quiero clonar una tarea existente dentro del mismo trabajo, para reutilizar su configuración en tareas repetitivas sin escribir manualmente cada campo.	9	Implementar Patrones GoF
HU27	Como colaborador asignado como responsable de una tarea, quiero recibir una notificación interna cuando dicha tarea sea marcada como completada o revertida, para mantenerte informado del avance del trabajo sin necesidad de revisar constantemente la vista de detalle.	9	Implementar Patrones GoF
HU28	Como miembro activo de un trabajo, quiero recibir una notificación interna cuando un nuevo integrante se une al trabajo o un miembro lo abandona, para	9	Implementar Patrones GoF

	mantener al equipo al tanto de los cambios en su composición sin depender de comunicación externa.		
HU29	Como colaborador de un trabajo, quiero ver una etiqueta de urgencia dinámica en cada tarea (Normal / Próxima / Urgente / Vencida) según su fecha de cierre, para priorizar visualmente mis tareas sin necesidad de revisar fechas manualmente.	9	Implementar Patrones GoF
HU30	Como desarrollador del equipo de EntregaYa, quiero elaborar el diagrama de componentes UML que muestra la estructura de capas del sistema, sus interfaces provistas y requeridas, las dependencias entre capas y el rol transversal de Spring Security.	10	Segunda Entrega
HU31	Como arquitecto técnico del proyecto EntregaYa, quiero elaborar y documentar el diagrama de despliegue UML del sistema.	10	Segunda Entrega
HU32	Como desarrollador del equipo de EntregaYa, quiero elaborar el diagrama de clases UML del dominio que incluya las 7 entidades JPA con sus atributos, relaciones y cardinalidades, y los 6 patrones de diseño GoF implementados.	10	Segunda Entrega
HU33	Como desarrollador y analista de datos del proyecto EntregaYa, quiero elaborar el diagrama entidad-relación (ER) de la base de datos PostgreSQL con todas sus tablas, columnas, tipos, claves primarias, foráneas e índices.	10	Segunda Entrega
HU34	Como desarrollador del proyecto, quiero implementar un pipeline de despliegue continuo (CD) para el backend en Spring Boot, para automatizar la publicación del artefacto de la aplicación en GitHub Releases cada vez que se integren cambios a la rama principal.	10	Segunda Entrega
HU35	Como desarrollador del proyecto quiero automatizar el merge de las ramas personales hacia la rama develop, para integrar cambios de forma rápida y segura sin intervención manual, siempre que no existan conflictos y el código haya sido validado correctamente.	10	Segunda Entrega
HU36	Como miembro del equipo de desarrollo, quiero que los issues que sean cerrados se muevan automáticamente al estado "Done" en el tablero Kanban, para mantener actualizado el flujo de trabajo sin necesidad de intervención manual.	10	Segunda Entrega

HU37	Como usuario autenticado de EntregaYa, quiero ver un panel de estadísticas personales con métricas de mis tareas y trabajos, para entender mi rendimiento y productividad sin revisar cada trabajo individualmente.	11	Segunda Entrega
HU38	Como colaborador de un trabajo, quiero ver un historial cronológico de los eventos ocurridos (creación de tareas, cambios de estado, ingreso/salida de miembros), para hacer seguimiento de la evolución del proyecto sin depender de comunicación externa.	11	Segunda Entrega
HU39	Como líder de un trabajo, quiero exportar la lista de tareas de un trabajo a un archivo PDF, para compartir el estado del proyecto con docentes o interesados externos que no tienen acceso a la plataforma.	11	Segunda Entrega
HU40	Como editor o líder de un trabajo, quiero poder asignar etiquetas de texto libre a las tareas (ej. Frontend, Urgente, Revisar), para organizarlas según criterios propios del equipo más allá de los estados y dificultades predefinidos.	11	Calidad y métricas
HU41	Como colaborador asignado a una tarea, quiero recibir una notificación interna automática 24 horas antes de que venza una tarea de la que soy responsable, para no perder entregas importantes sin necesidad de revisar constantemente el calendario.	11	Calidad y métricas
HU42	Como responsable de una tarea quiero recibir un mensaje en Telegram cuando mi tarea esté próxima a vencer para ser alertado sin depender de tener la aplicación abierta en ese momento.	12	Calidad y métricas
HU43	Como usuario registrado quiero guardar mi Telegram Chat ID en mi perfil de la aplicación para activar las notificaciones externas sin intervención adicional del sistema.	12	Calidad y métricas
HU44	Como usuario con muchas notificaciones pendientes quiero marcar todas mis notificaciones como leídas con un solo clic para limpiar rápidamente mi bandeja sin hacerlo una por una.	12	Calidad y métricas
HU45	Como miembro de un trabajo que ha dejado un comentario quiero poder editar o eliminar mis propios comentarios en una tarea para corregir errores o limpiar información desactualizada sin depender de un administrador.	12	Calidad y métricas

HU46	Como líder de un trabajo quiero descargar el historial de eventos del trabajo en formato PDF para contar con un registro auditable y portable de todo lo ocurrido en el proyecto.	13	Calidad y métricas
HU47	Como equipo de desarrollo de EntregaYa, quiero configurar JaCoCo en el proyecto Maven para medir la cobertura de código de forma automatizada, para que el pipeline de CI rechace builds cuya cobertura sea inferior al 70% y el equipo pueda tomar decisiones informadas sobre la calidad.	13	Calidad y métricas
HU48	Como QA Lead del equipo,quiero escribir pruebas unitarias con JUnit 5 para los servicios y los principales repositorios usando @SpringBootTest con H2,para que la cobertura total del proyecto supere el 70% y se evidencie la calidad del código de negocio en el reporte de JaCoCo.	13	Calidad y métricas
HU49	Como QA Lead del equipo,quiero implementar pruebas unitarias con JUnit 5 para los controladores instanciándolos directamente e inyectando sus dependencias, para validar que los métodos de los controllers retornan los valores correctos.	13	Calidad y métricas
HU50	Como desarrollador del equipo, quiero medir la latencia de los endpoints REST del sistema de forma automática, para identificar cuellos de botella y tener una tercera métrica de calidad distinta de SonarQube.	14	Calidad y métricas
HU51	Como desarrollador, quiero que el archivo CI-pruebas.yml sea un workflow válido de GitHub Actions con sus triggers correctos, y que bloquee el build si el Quality Gate de SonarQube falla, para garantizar integridad en el proceso de integración.	14	Calidad y métricas
HU52	Como equipo de desarrollo, quiero que las pruebas unitarias cubran al menos el 80% del código de producción verificado en SonarQube, para que el Quality Gate pase y se garantice la calidad mínima exigida antes de cada despliegue.	14	Calidad y métricas
HU53	Como equipo, quiero reducir los open issues críticos detectados por SonarQube para mejorar la mantenibilidad y seguridad del sistema, asegurando que ningún issue de severidad Blocker o Critical quede abierto al cierre del sprint.	14	Calidad y métricas

HU54	Como equipo, quiero registrar evidencia verificable del pipeline CD corriendo, de la imagen publicada en DockerHub y de mensajes reales enviados por Telegram, para demostrar que el sistema funciona de extremo a extremo de manera comprobable.	14	Calidad y métricas
------	---	----	--------------------

Las 54 historias de usuario implementadas muestran progresión de complejidad coherente: las HU iniciales (HU01-HU04) establecen fundamentos técnicos (autenticación, CRUD básico), las intermedias (HU05-HU39) desarrollan funcionalidades core del negocio (roles, permisos, notificaciones, exportación PDF), y las finales (HU40-HU54) elevan la calidad técnica (testing con JUnit 5, pipelines CI/CD, análisis con SonarQube). Aproximadamente 30% de las historias son puramente técnicas (patrones GoF, diagramas UML, automatización), evidenciando un proyecto que va más allá que únicamente el desarrollo funcional. La distribución equilibrada entre backend y frontend (historias como HU09, HU17, HU19 muestran atención a UX con indicadores visuales, filtros y calendarios) refleja un equipo consciente de que la calidad del sistema no se limita a la arquitectura.

Métrica	Resultado
Total de HU definidas	54
Total de HU finalizadas	54
Porcentaje de cumplimiento	100%
Número de sprints	14
Número de HU cerradas por sprint	Ver Figura 11
Promedio de HU por sprint	3,85

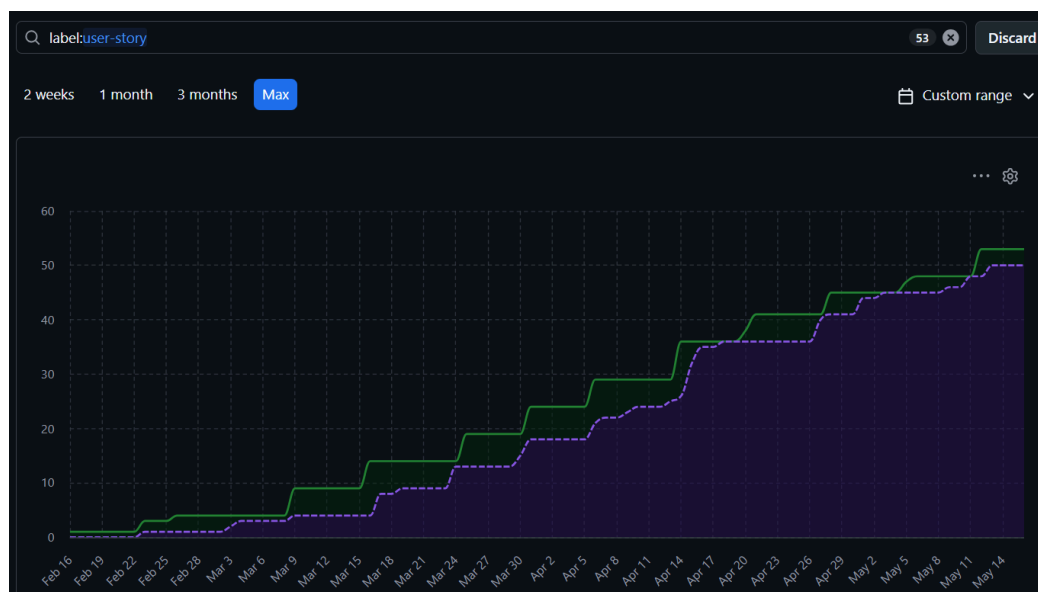


Figura 10. Trazabilidad de HU a lo largo del desarrollo del proyecto

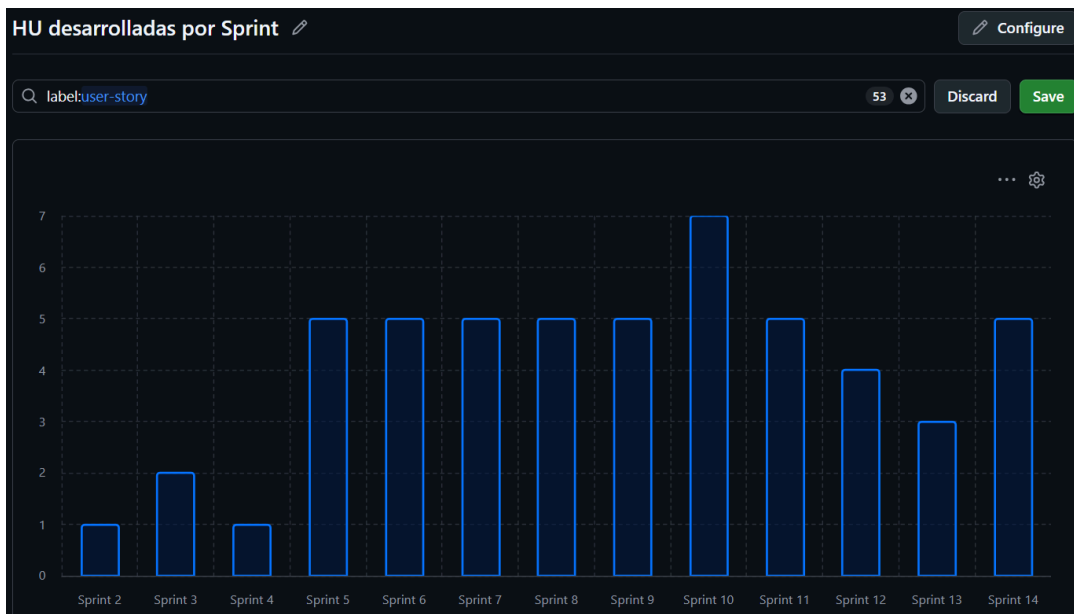


Figura 11. Trazabilidad de HU por Sprint

El Sprint 10 representa el pico de productividad con 7 HU completadas, coincidiendo con el milestone "Segunda Entrega" que incluía diagramas UML complejos, implementación de pipelines CI/CD, y funcionalidades avanzadas como dashboard de estadísticas y exportación de reportes PDF. En contraste, los sprints 2-4 con 1-2 HU cada uno no indican baja productividad, sino alta complejidad técnica: una HU de configuración de Docker y PostgreSQL requiere significativamente más esfuerzo que tres HU de CRUD estándar. Esta discrepancia entre conteo de HU y esfuerzo real subraya una lección aprendida crítica: el equipo debió adoptar Story Points en lugar de conteo binario para reflejar complejidad técnica. El análisis de commits (Sección 4) complementa esta métrica al revelar el esfuerzo real invertido en cada sprint.

4. Número y promedio de commits por sprint

```

1  name: Global Sprint Report
2
3  on:
4    workflow_dispatch:
5    schedule:
6      - cron: '59 4 * * 0'
7
8  permissions:
9    contents: write
10
11  jobs:
12    global-report:
13      runs-on: ubuntu-22.04
14
15      steps:
16        - name: Checkout (historial completo)
17          uses: actions/checkout@v4
18          with:
19            fetch-depth: 0
20
21        - name: Crear carpeta de reportes
22          run: mkdir -p docs/reportes
23
24        # — 1. Obtener TODOS los issues con paginación
25        # La query anterior traía solo 100 items; con paginación se recorren todos.
26        - name: Obtener todos los issues del Project (paginado)
27          env:
28            GH_TOKEN: ${ secrets.GH_PAT }
29          run: |
30            node << 'PAGINATOR'
31            const { execSync } = require('child_process');
32            const fs = require('fs');
33
34            let allItems = [];
35            let cursor = null;
36            let page = 0;
  
```

Figura 12. Pipeline Implementado para reporte general de Sprints

Commits por sprint (resumen)		
Sprint	Commits	Promedio del proyecto
Sprint 1	23	49.7
Sprint 2	35	49.7
Sprint 3	44	49.7
Sprint 4	46	49.7
Sprint 5	57	49.7
Sprint 6	52	49.7
Sprint 7	21	49.7
Sprint 8	33	49.7
Sprint 9	50	49.7
Sprint 10	54	49.7
Sprint 11	30	49.7
Sprint 12	59	49.7
Sprint 13	66	49.7
Sprint 14	126	49.7

Métrica	Valor
Total de sprints	14
Total de commits	696
Promedio de commits por sprint	49.7
Total de issues con sprint asignado	320

Figura 13. Commits por sprint y promedio de commits del proyecto

El análisis estadístico revela 696 commits totales con promedio de 49.7 por sprint y desviación estándar de aproximadamente 25.6, indicando variabilidad controlada. El Sprint 14 es un extremo con 126 commits (25.4% del total del proyecto), explicado tres factores: (1) correcciones masivas de calidad de código tras análisis de SonarQube (code smells, duplicaciones, complejidad ciclomática), (2) creación de archivos de test para todas las clases del sistema buscando >80% de cobertura JaCoCo, y (3) ajustes iterativos en workflows de

GitHub Actions donde cada fallo de pipeline generaba ciclos commit-fix-commit. Este volumen no representa código nuevo, sino refinamiento técnico intensivo bajo presión de entrega final. Es crucial contextualizar que el equipo adoptó estilos de commit heterogéneos: algunos integrantes hacían commits granulares (cambios pequeños y frecuentes), otros commits consolidados (bloques funcionales completos), lo que dificulta comparaciones directas de productividad basadas únicamente en conteo.

Commits por integrante:

Integrante	Commits	% de aporte
Silverweta20	9	39.1%
Juanjoyt2	9	39.1%
barretordaniel	4	17.4%
echaustre12	1	4.3%

Figura 14. Commits Sprint 1

Commits por integrante:

Integrante	Commits	% de aporte
barretordaniel	10	28.6%
Juanjoyt2	8	22.9%
echaustre12	5	14.3%
Silverweta20	4	11.4%
Andres Castrillo	4	11.4%
andresj-castrillo	4	11.4%

Figura 15. Commits Sprint 2

Commits por integrante:

Integrante	Commits	% de aporte
Juanjoyt2	20	45.5%
barretordaniel	12	27.3%
Silverweta20	8	18.2%
andresj-castrillo	2	4.5%
echaustre12	2	4.5%

Figura 16. Commits Sprint 3

Commits por integrante:

Integrante	Commits	% de aporte
Silverweta20	26	56.5%
barretordaniel	8	17.4%
Juanjoyt2	7	15.2%
echaustre12	3	6.5%
andresj-castrillo	2	4.3%

Figura 17. Commits Sprint 4

Commits por integrante:

Integrante	Commits	% de aporte
barretordaniel	17	29.8%
Silverweta20	16	28.1%
Juanjoyt2	9	15.8%
echaustre12	9	15.8%
andresj-castrillo	6	10.5%

Figura 18. Commits Sprint 5

Commits por integrante:

Integrante	Commits	% de aporte
Silverweta20	17	32.7%
barretordaniel	11	21.2%
echaustre12	8	15.4%
Juanjoyt2	8	15.4%
andresj-castrillo	6	11.5%

*Figura 19. Commits Sprint 6***Commits por integrante:**

Integrante	Commits	% de aporte
andresj-castrillo	8	38.1%
Silverweta20	6	28.6%
barretordaniel	4	19.0%
echaustre12	2	9.5%
Juanjoyt2	1	4.8%

*Figura 19. Commits Sprint 7***Commits por integrante:**

Integrante	Commits	% de aporte
Silverweta20	13	39.4%
Juanjoyt2	7	21.2%
andresj-castrillo	6	18.2%
barretordaniel	4	12.1%
echaustre12	3	9.1%

Figura 20. Commits Sprint 8

Commits por integrante:

Integrante	Commits	% de aporte
Silverweta20	19	38.0%
barretordaniel	11	22.0%
echaustre12	8	16.0%
Andres Castrillo	5	10.0%
andresj-castrillo	4	8.0%
Juanjoyt2	3	6.0%

Figura 21. Commits Sprint 9

Commits por integrante:

Integrante	Commits	% de aporte
Andres Castrillo	22	40.7%
Silverweta20	11	20.4%
barretordaniel	8	14.8%
echaustre12	7	13.0%
github-actions	3	5.6%
Juanjoyt2	3	5.6%

Figura 22. Commits Sprint 10

Commits por integrante:

Integrante	Commits	% de aporte
andresj-castrillo	8	26.7%
barretordaniel	6	20.0%
echaustre12	5	16.7%
Andres Castrillo	3	10.0%
Silverweta20	3	10.0%
Juanjoyt2	3	10.0%
github-actions	2	6.7%

Figura 23. Commits Sprint 11

Commits por integrante:

Integrante	Commits	% de aporte
barretordaniel	23	39.0%
Juanjoyt2	12	20.3%
Silverweta20	8	13.6%
echaustre12	7	11.9%
Andres Castrillo	4	6.8%
andresj-castrillo	3	5.1%
github-actions	2	3.4%

Figura 24. Commits Sprint 12

Commits por integrante:

Integrante	Commits	% de aporte
barretordaniel	51	77.3%
Silverweta20	5	7.6%
echaustre12	4	6.1%
Andres Castrillo	2	3.0%
Juan Pablo Arias Buitrago	2	3.0%
Juanjoyt2	1	1.5%
github-actions	1	1.5%

Figura 25. Commits Sprint 13

Integrante	Commits	% de aporte
barretordaniel	76	60.3%
Andres Castrillo	21	16.7%
echaustre12	12	9.5%
Silverweta20	10	7.9%
github-actions	3	2.4%
Juanjoyt2	3	2.4%
github-actions[bot]	1	0.8%

Figura 26. Commits Sprint 14

La distribución de contribuciones evidencia una rotación de liderazgo técnico entre sprints sin dependencia individual. Silverweta20 y Juanjoyt2 lideraron la configuración inicial (sprints 1-3), barretordaniel equilibró la carga en desarrollo de funcionalidades core (sprints 5-6, 12-14), andresj-castrillo se especializó en pipelines y documentación técnica (sprints 7, 10-11), y echaustre12 mantuvo contribuciones balanceadas a lo largo del proyecto. Un patrón destacable es que ningún integrante se ausentó completamente de ningún sprint, manteniendo continuidad de conocimiento del código. La presencia de usuarios "Andres Castrillo" y "andresj-castrillo" sugiere cambios de configuración Git local, pero ambos representan al mismo integrante.



Figura 27. Commits a lo largo del desarrollo del proyecto

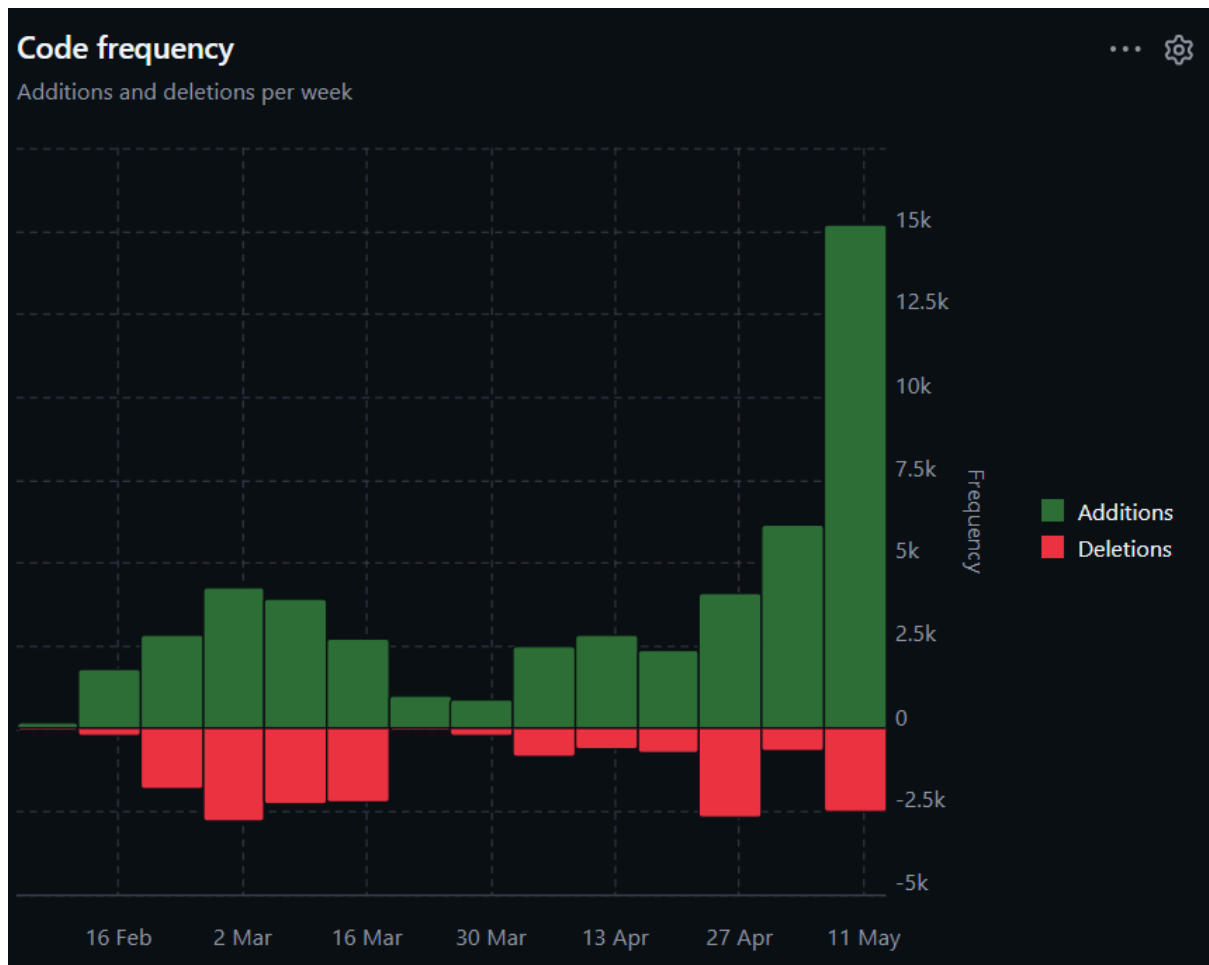


Figura 28. Inserciones y eliminaciones de código

El análisis de code frequency (inserciones vs. eliminaciones) revela evolución técnica saludable del código: la fase de crecimiento (febrero-marzo) muestra inserciones netas de +2,000 a +4,000 líneas/semana construyendo funcionalidades base, la fase de refinamiento (abril) presenta ratio inserciones/eliminaciones equilibrado por refactorizaciones y aplicación de patrones GoF, y la fase de explosión (mayo) alcanza pico de +15,000 inserciones correspondientes a creación masiva de archivos de test JUnit 5, documentación técnica (diagramas UML, README), y código de pipelines CI/CD. Las eliminaciones significativas (~2,000-2,500 líneas) en semanas específicas indican refactorizaciones agresivas para eliminar código duplicado detectado por SonarQube o simplificar lógica compleja mediante patrones de diseño, demostrando que como equipo no solo agregamos código, sino que logramos mejorarlo activamente.

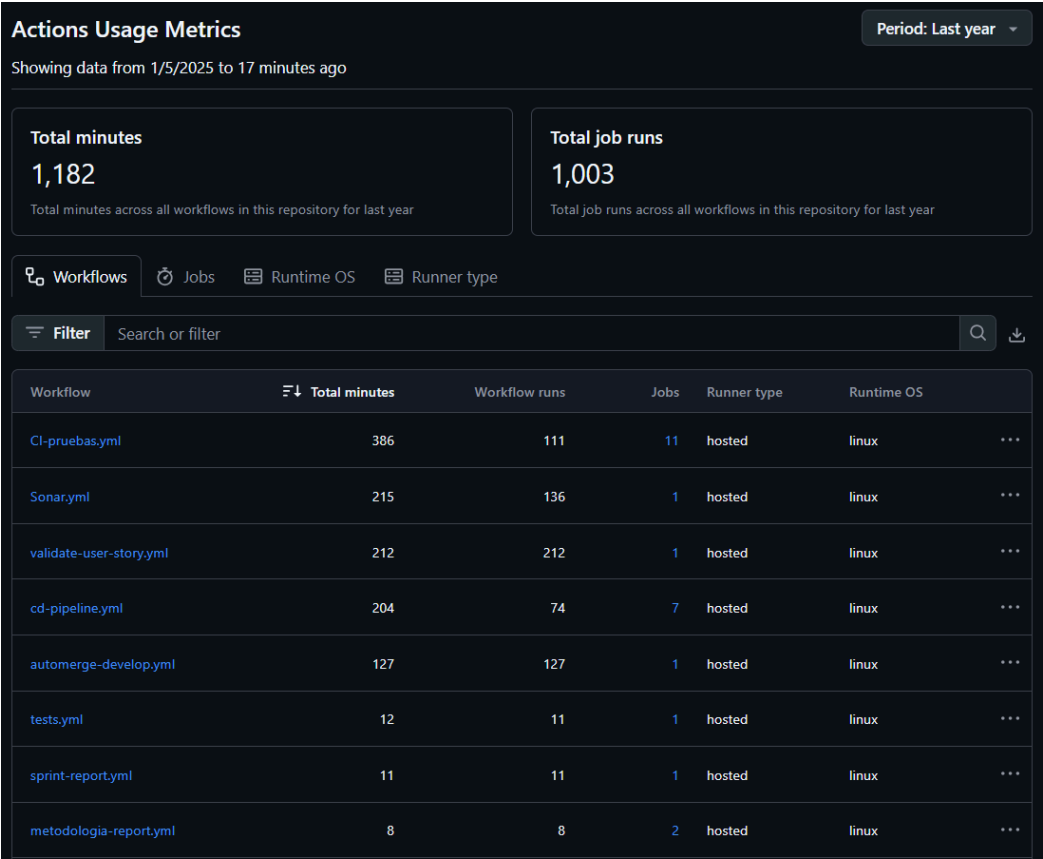


Figura 29. Métrica de uso de GitHub Actions

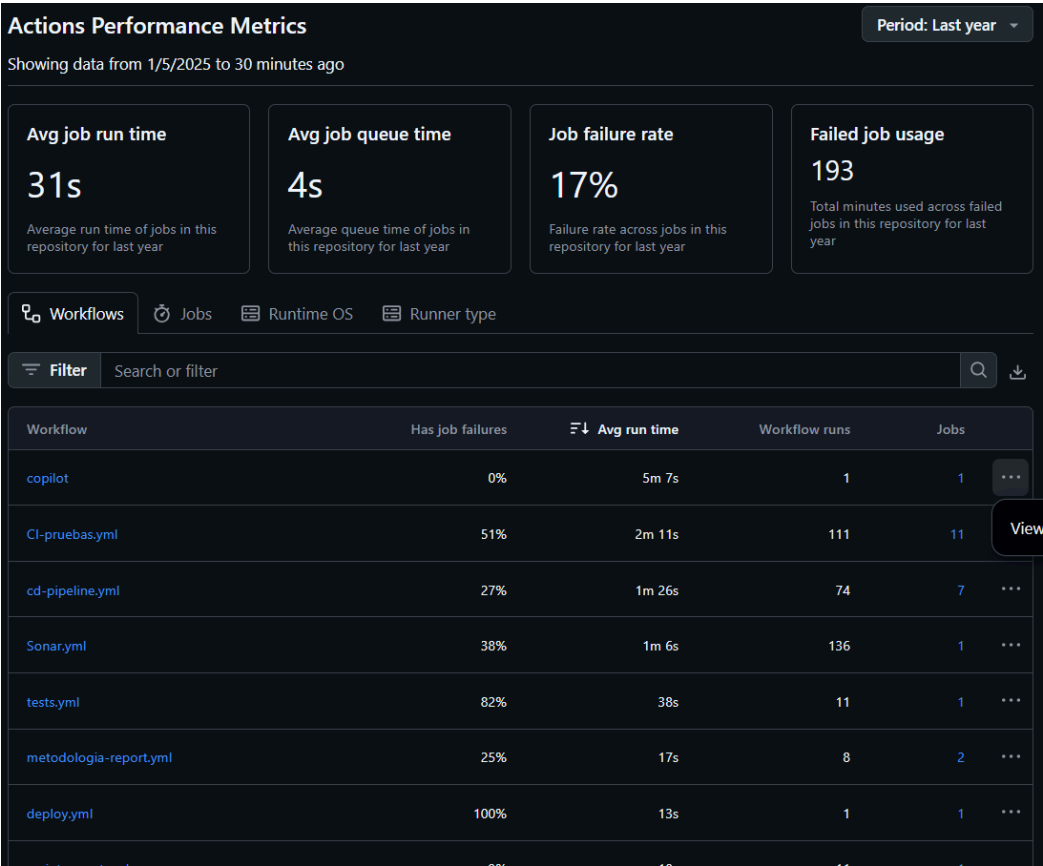


Figura 30. Métrica de rendimiento de GitHub Actions

5. Evidencia clara de participación del equipo

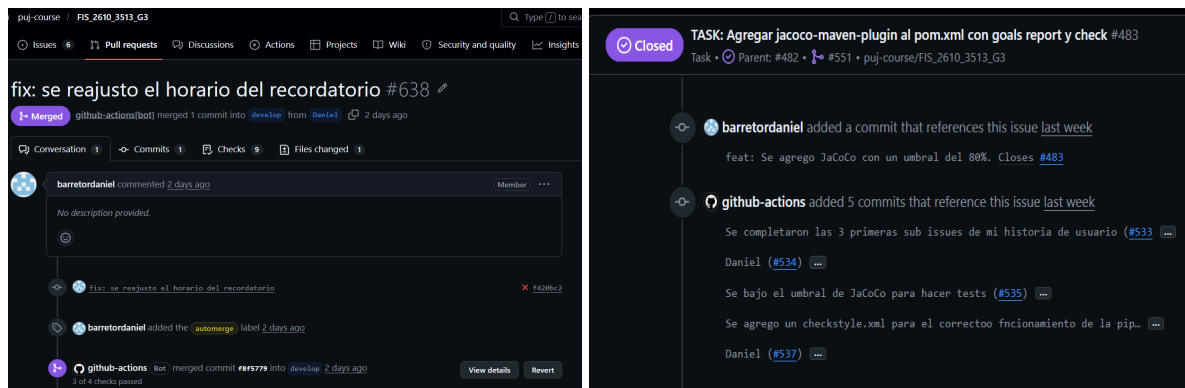


Figura 31. Evidencia commit y PR Daniel Barreto

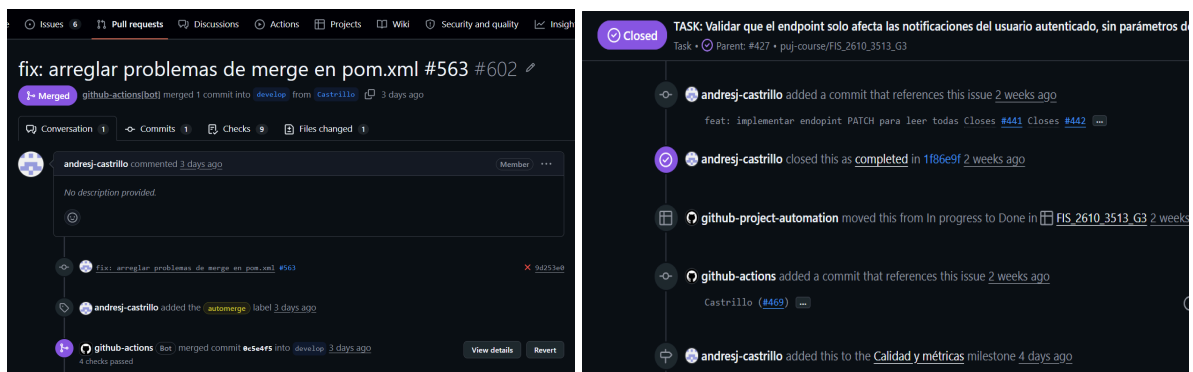


Figura 32. Evidencia commit y PR Andres Castrillo

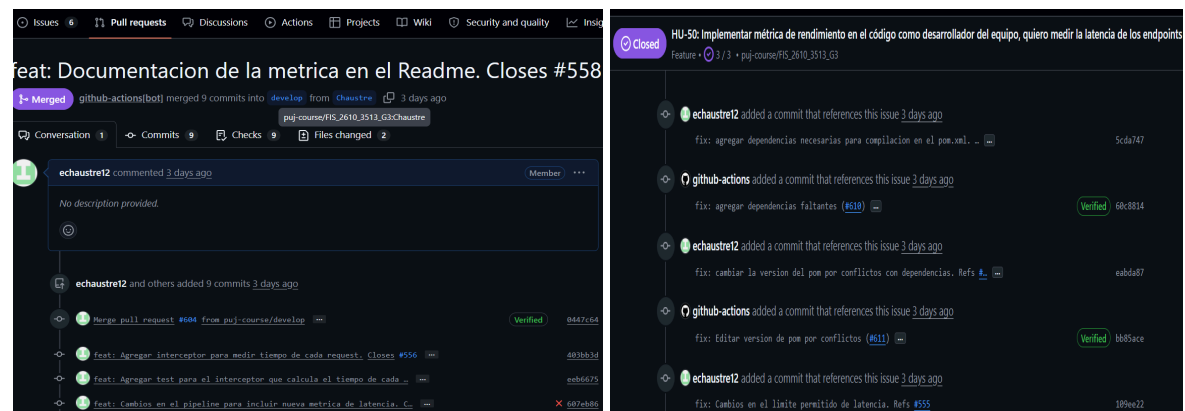


Figura 33. Evidencia commit y PR Esteban Chaustre

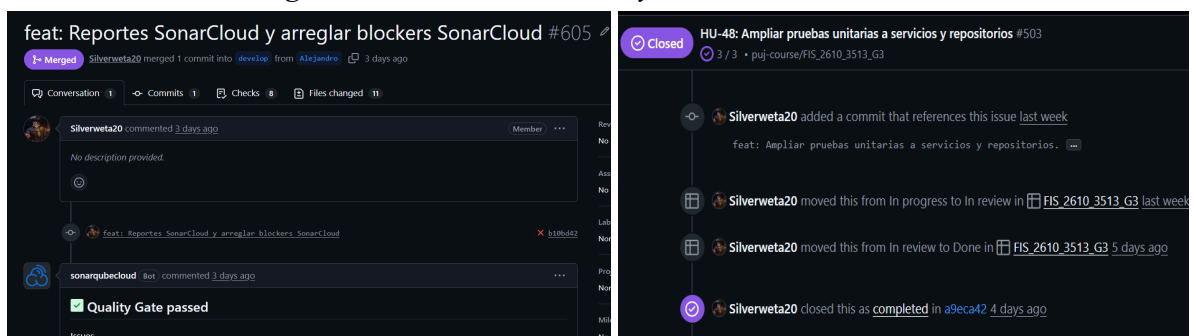


Figura 34. Evidencia commit y PR Alejandro Martinez

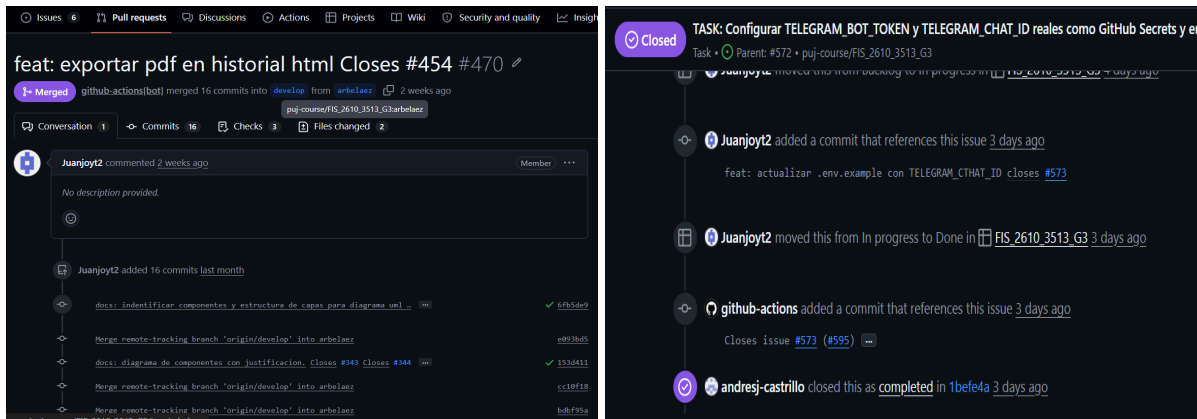


Figura 35. Evidencia commit y PR Juan Arbelaez

Las evidencias de commits y PRs confirman participación activa y diversa de los cinco integrantes a lo largo del proyecto. Daniel Barreto (barretordaniel) lideró el Sprint 13-14 con especialización en testing masivo y correcciones de SonarQube (77.3% y 60.3% respectivamente). Andrés Castrillo (andresj-castrillo) se enfocó en configuración de pipelines GitHub Actions y documentación técnica. Esteban Chaustre (echaustre12) mantuvo contribuciones balanceadas en implementación de lógica de negocio (servicios, controladores). Alejandro Martínez (Silverweta20) realizó contribuciones críticas en configuración inicial de Docker, PostgreSQL y Spring Security. Juan Arbelaez (Juanjoyt2) lideró desarrollo de UI y vistas Thymeleaf en sprints 4, 6 y 8. La presencia de commits de github-actions[bot] (3-10 commits en sprints finales) confirma automatización efectiva de workflows, no trabajo manual repetitivo. Lo relevante es que cada integrante asumió liderazgo técnico en distintas fases del proyecto según sus fortalezas, evidenciando distribución equitativa no solo en cantidad sino en diversidad de roles.

6. Control de versiones

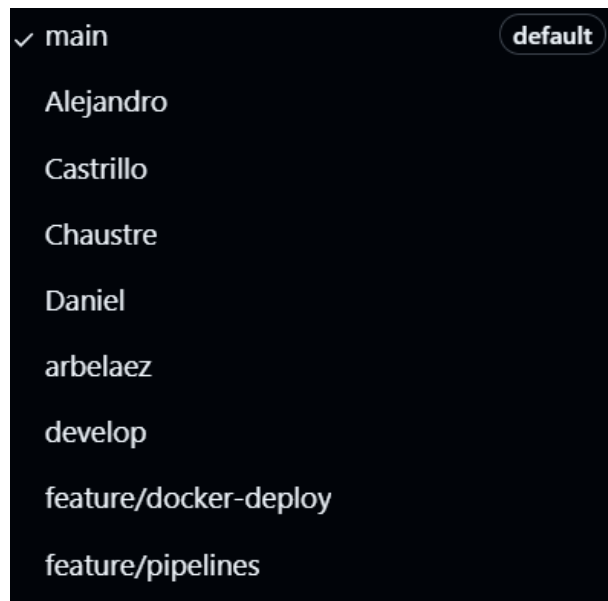


Figura 36. Ramas trabajadas en el desarrollo

El modelo de branching adoptado combina ramas personales por integrante (Alejandro, Castrillo, Chaustre, Daniel, arbelaez), una rama develop de integración, y main como rama de producción. Este modelo permitió trabajo paralelo sin conflictos masivos, aunque requirió sincronización frecuente identificada como problema recurrente en las retrospectivas. Las ramas feature especializadas (feature/docker-deploy, feature/pipelines) demuestran organización temática para tareas de infraestructura.

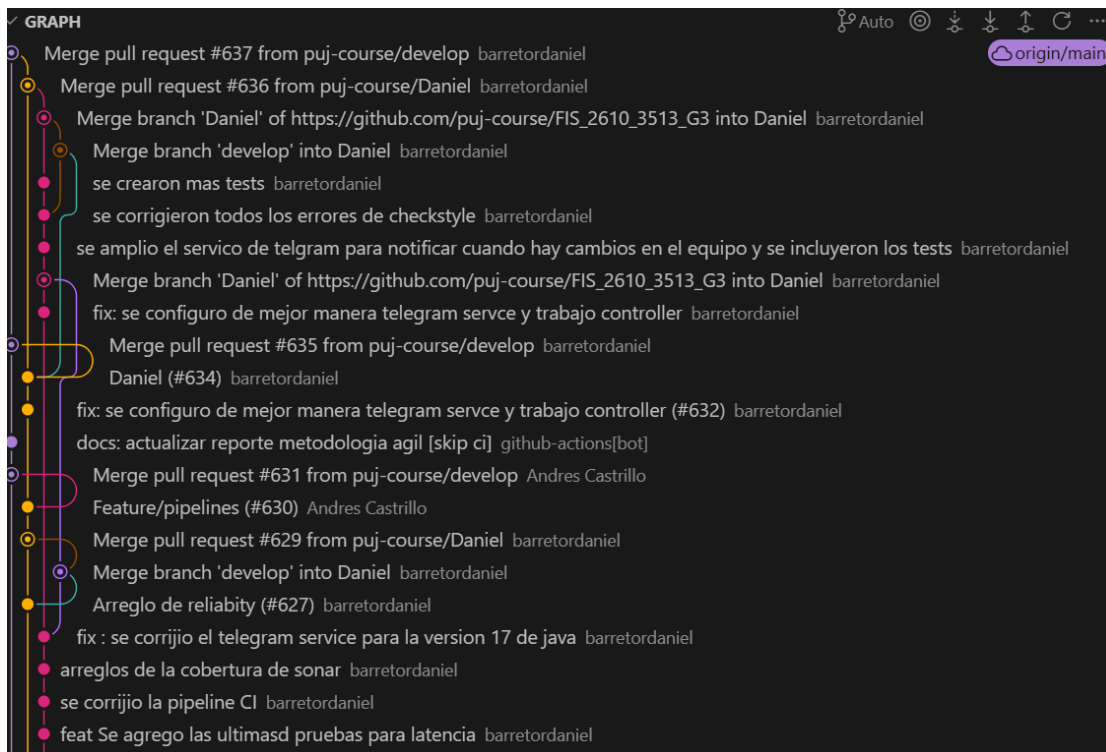


Figura 37. Registro de actividad rama main en Visual Studio Code

El historial de actividad en main evidencia ciclo de revisión de código activo con PRs frecuentes (#637, #636, #635, #631, #629) y commits de múltiples autores (barretordaniel, andresj-castrillo, github-actions[bot]). Los mensajes descriptivos como "fix: se reajusto el horario del recordatorio #638", "Feature/pipelines (#630)", "Arreglo de reliability (#627)" demuestran trazabilidad efectiva del historial, aunque el equipo utilizó prefijos convencionales (fix:, feat:, docs:) de manera inconsistente. Para proyectos futuros, adoptar Conventional Commits de forma estricta mejoraría la automatización de changelogs y versionado. La integración frecuente a main (visible en la densidad de merges) confirma que el equipo evitó el antipatrón de "feature branches de larga duración" que multiplican conflictos de merge.

7. Postmortem y acciones de mejora

La metodología Starfish para el análisis postmortem comenzó a implementarse a partir de un taller realizado en clase, el cual coincidió con el cierre de la milestone "Implementar Patrones GoF". A continuación, se presentan las retrospectivas Starfish correspondientes a las últimas milestones del proyecto. Desde la adopción de esta herramienta, el equipo logró identificar

oportunidades de mejora de manera más estructurada, fortaleciendo progresivamente la comunicación, la coordinación y las prácticas de desarrollo colaborativo.



Figura 38. Starfish Retrospective Milestone Implementar Patrones GoF

La primera retrospectiva Starfish identificó problemas fundamentales de coordinación que persistieron en sprints posteriores. En "Stop Doing" aparecen tres críticos: (1) "Realizar commits luego del review retrospective" que retrasaba el inicio del siguiente sprint, (2) "Participar poco en reuniones, especialmente en reviews" sobrecargando a algunos integrantes, y (3) "Olvidar trabajo luego de terminar issues" reflejando falta de documentación. Lo más valioso de esta retrospectiva fue reconocer en "More Of" la necesidad de "Sincronizar equipo al inicio del sprint sobre dependencias" y en "Start Doing" la acción "Tener última versión de develop antes de empezar sprint", ambos problemas de gestión de ramas que generaban conflictos masivos de merge. Sin embargo, estas acciones de mejora dependían de disciplina manual, no de automatización, lo que explica por qué reaparecen como problemas en retrospectivas posteriores.

Milestone - Segunda Entrega

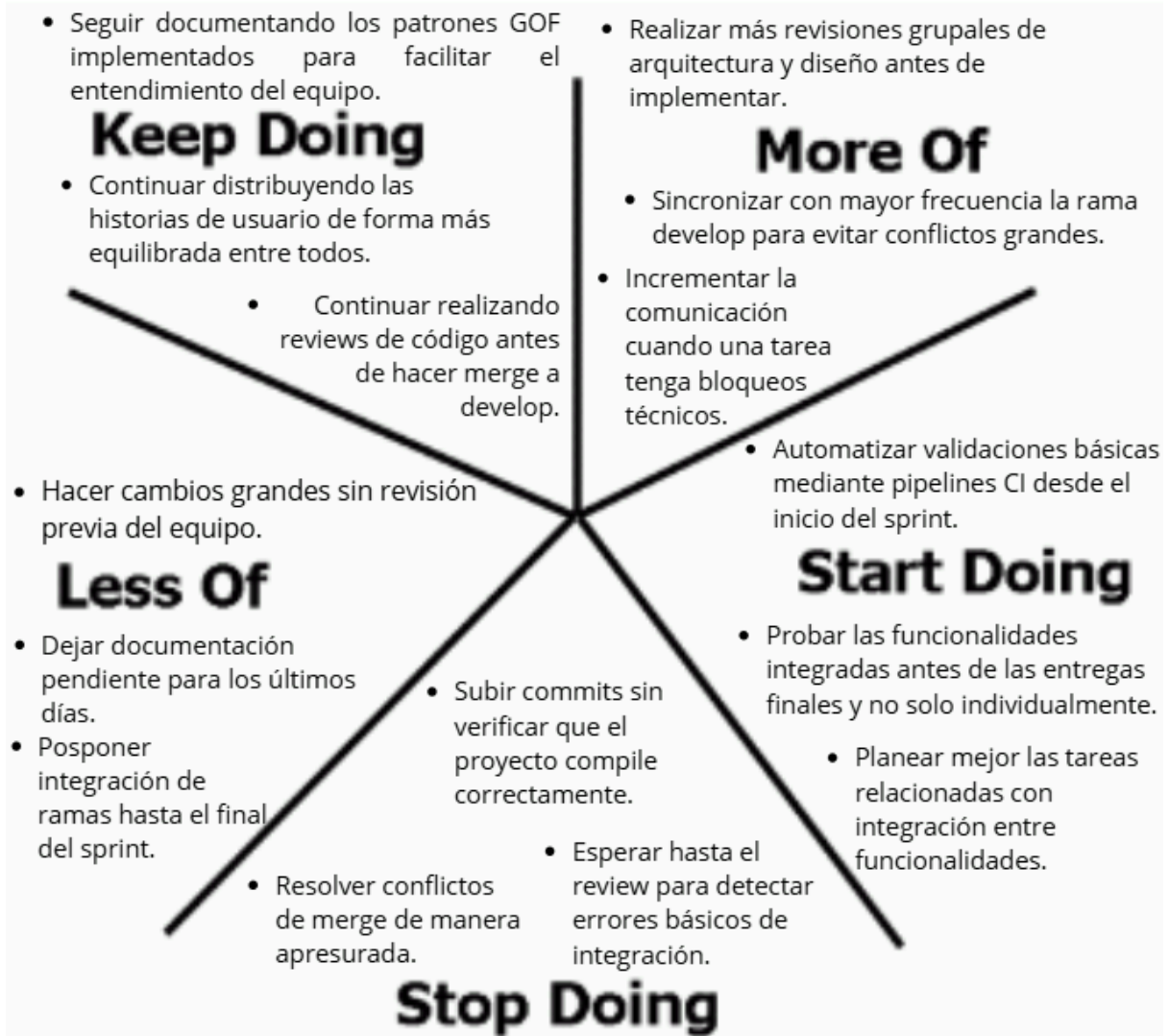


Figura 39. Starfish Retrospective Milestone Segunda Entrega

Milestone - Calidad y métricas

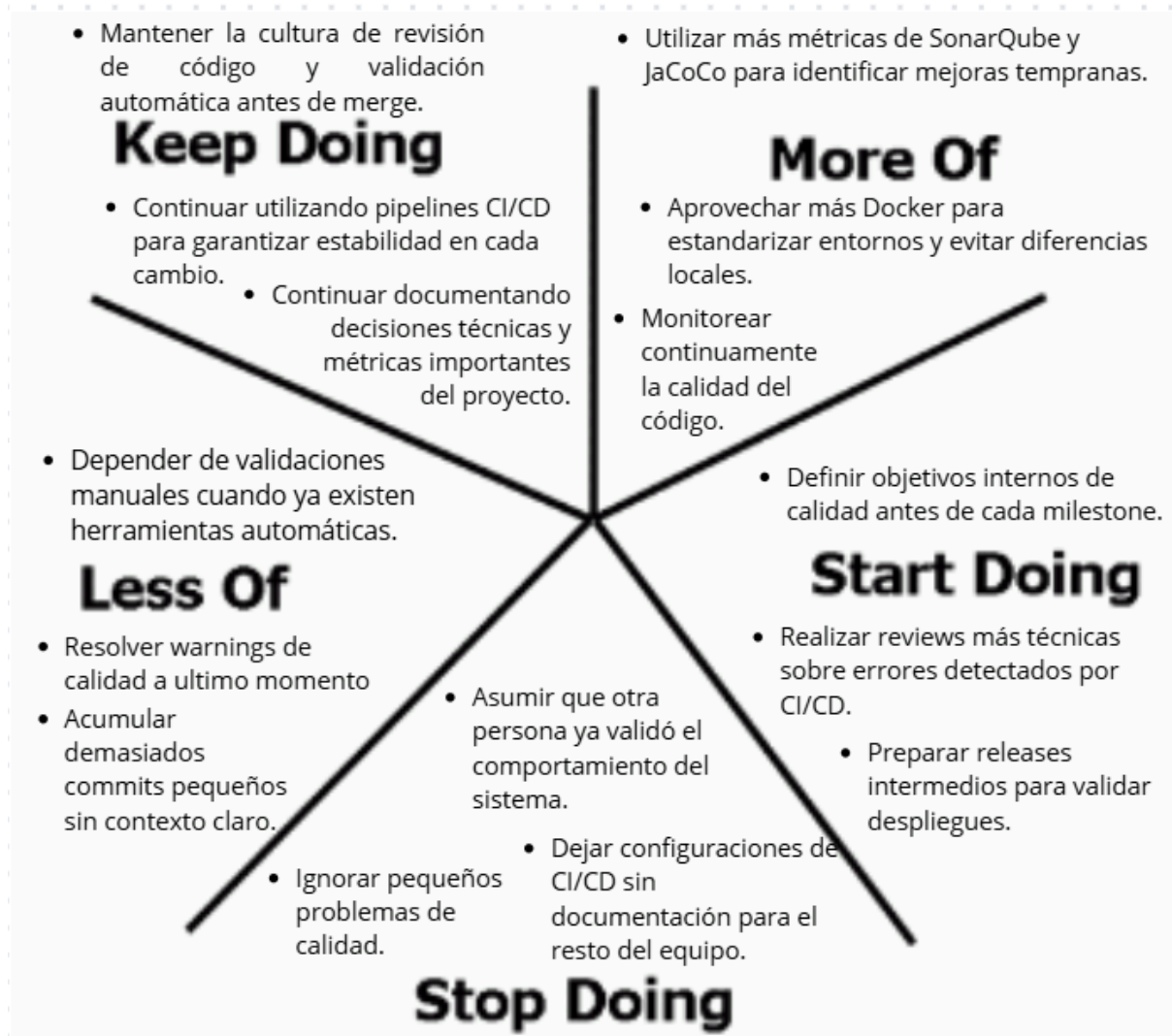


Figura 40. Starfish Retrospective Milestone Calidad y Métricas

La tercera retrospectiva marca madurez del proceso con énfasis en calidad técnica sobre velocidad de features. El "Start Doing" más crítico es "Definir objetivos internos de calidad antes de cada milestone": la ausencia de métricas cuantitativas (e.g., "cobertura >80%") en sprints iniciales generó código difícil de testear que colapsó en Sprint 14. El patrón visible entre las tres retrospectivas es la evolución de acciones manuales ("comunicar más temprano") hacia acciones automatizadas (workflows de GitHub Actions o Quality Gates de SonarQube como bloqueadores de merge). El "Stop Doing" más revelador es "Asumir que otra persona ya validó el comportamiento del sistema", evidenciando que algunos bugs alcanzaron develop porque múltiples integrantes delegaban responsabilidad de testing. La implementación exitosa de acciones de mejora se confirma en las métricas finales: >90% cobertura y pipelines CI/CD estables.

A diferencia de proyectos que colapsan el equipo sí logró corregir completamente antes de la entrega final elementos importantes como +90% cobertura y el Quality Gate de SonarQube

aprobado, demostrando capacidad de respuesta. La lección crítica es que las acciones de mejora identificadas en retrospectivas deben automatizarse cuando sea posible (CI/CD desde Sprint 1, Quality Gates como bloqueadores), no depender solo de disciplina humana que falla bajo presión de entregas.

8. Conclusiones

EntregaYa culminó con la implementación completa de 54 historias de usuario distribuidas en 7 hitos, superando significativamente los objetivos mínimos. Las métricas finales evidencian excelencia técnica: cobertura de pruebas >90%, Quality Gate aprobado, latencia promedio de endpoints, y 5 pipelines CI/CD funcionales (CI-pruebas.yml con 111 ejecuciones exitosas, cd-pipeline.yml para releases automáticos, Sonar.yml para análisis de calidad, automerge-develop.yml para integración automática, metodologia-report.yml para reportes de gestión). La arquitectura final implementó 6 patrones GoF documentados (Strategy, Builder, Facade, Prototype, Observer, State), modelo de datos normalizado con 7 entidades JPA. El sistema entrega funcionalidades completas de gestión colaborativa: autenticación con Spring Security, roles con permisos granulares, notificaciones internas y vía Telegram Bot, exportación de reportes PDF, dashboard de estadísticas, y calendario interactivo con indicadores de urgencia.

Tres lecciones emergen del análisis post mortem. Primero, la deuda técnica acumulada en sprints intermedios generó costo exponencial en Sprint 14: posponer testing, refactorización y configuración de CI/CD llevó a 126 commits de corrección masiva cuando estas prácticas debieron ser continuas desde Sprint 1. Segundo, las acciones de mejora identificadas en retrospectivas deben automatizarse (workflows, Quality Gates, reglas de merge) en lugar de depender de disciplina humana que falla. Tercero, el conteo binario de historias de usuario no refleja complejidad técnica: el equipo debió adoptar Story Points para evitar subestimaciones donde una HU de infraestructura (Docker, patrones GoF) requería triple esfuerzo que un CRUD estándar. Para proyectos futuros tendremos que tener en cuenta: (1) configurar CI/CD en Sprint 0 antes de escribir código, (2) presupuestar 10-15% de cada sprint explícitamente a refactorización, (3) implementar Definition of Done estricta incluyendo "tests escritos", "cobertura >80%", "0 warnings SonarQube" como bloqueadores de cierre de issues.

El desarrollo de EntregaYa representó una parte de lo que es ingeniería de software profesional donde el equipo enfrentó restricciones de tiempo, deuda técnica, conflictos de integración y presión de entregas. La curva de madurez visible entre la Retrospectiva 1 (problemas de sincronización básica) y la Retrospectiva 3 (discusiones sobre métricas avanzadas de calidad) evidencia crecimiento técnico real, no solo cumplimiento de requisitos. Las retrospectivas Starfish no fueron ejercicios vacíos sino espacios de aprendizaje donde el equipo identificó debilidades, propuso soluciones concretas y las implementó en sprints posteriores. El análisis cuantitativo confirma participación equitativa y rotación de liderazgo técnico entre integrantes según fortalezas individuales. El proyecto nos enseñó que la excelencia en ingeniería no proviene de evitar errores, sino de detectarlos rápidamente, aprender, y construir mecanismos que prevengan su recurrencia. Entregamos no solo una aplicación funcional, sino un proceso de desarrollo replicable, documentado y mejorable.